UNIVERSITY OF ASIA PACIFIC
DEPARTMENT OF CSE

Lab Report - 6

COURSE CODE: CSE 402.

COURSE TITLE: OPERATING SYSTEM LAB.

SUBMITTED BY:

NAME : SAZID ABDULLAH

STUDENT ID : 23101146

SEMESTER : 4th YEAR 1st SEMESTER

SECTION : C - 2

DATE : 9/2/2026

SUBMITTED TO

ATIA RAHMAN ORTHI

LECTURER AT UAP

1. Problem:

Given the following processes with their **Arrival Time (AT)**, **Burst Time (BT)**, and **Priority**:

Process	Arrival Time (AT)	Burst Time (BT)	Priority
P1	0	3	3
P2	1	4	2
P3	2	6	4
P4	3	4	6
P5	5	2	10

Implement the following CPU Scheduling Algorithms:

- **Priority Non-Preemptive Scheduling**
- **Priority Preemptive Scheduling**

Assume that a **smaller priority number indicates a higher priority**.

For each algorithm, calculate the following for every process:

- **Completion Time (CT)**
- **Turnaround Time (TAT)**
- **Waiting Time (WT)**

Also, calculate:

- **Average Turnaround Time (Average TAT)**
- **Average Waiting Time (Average WT)**

Finally, display the results for all processes and **compare the Average TAT and Average WT of both Preemptive and Non-Preemptive Priority Scheduling algorithms to determine which algorithm performs better for the given processes.**

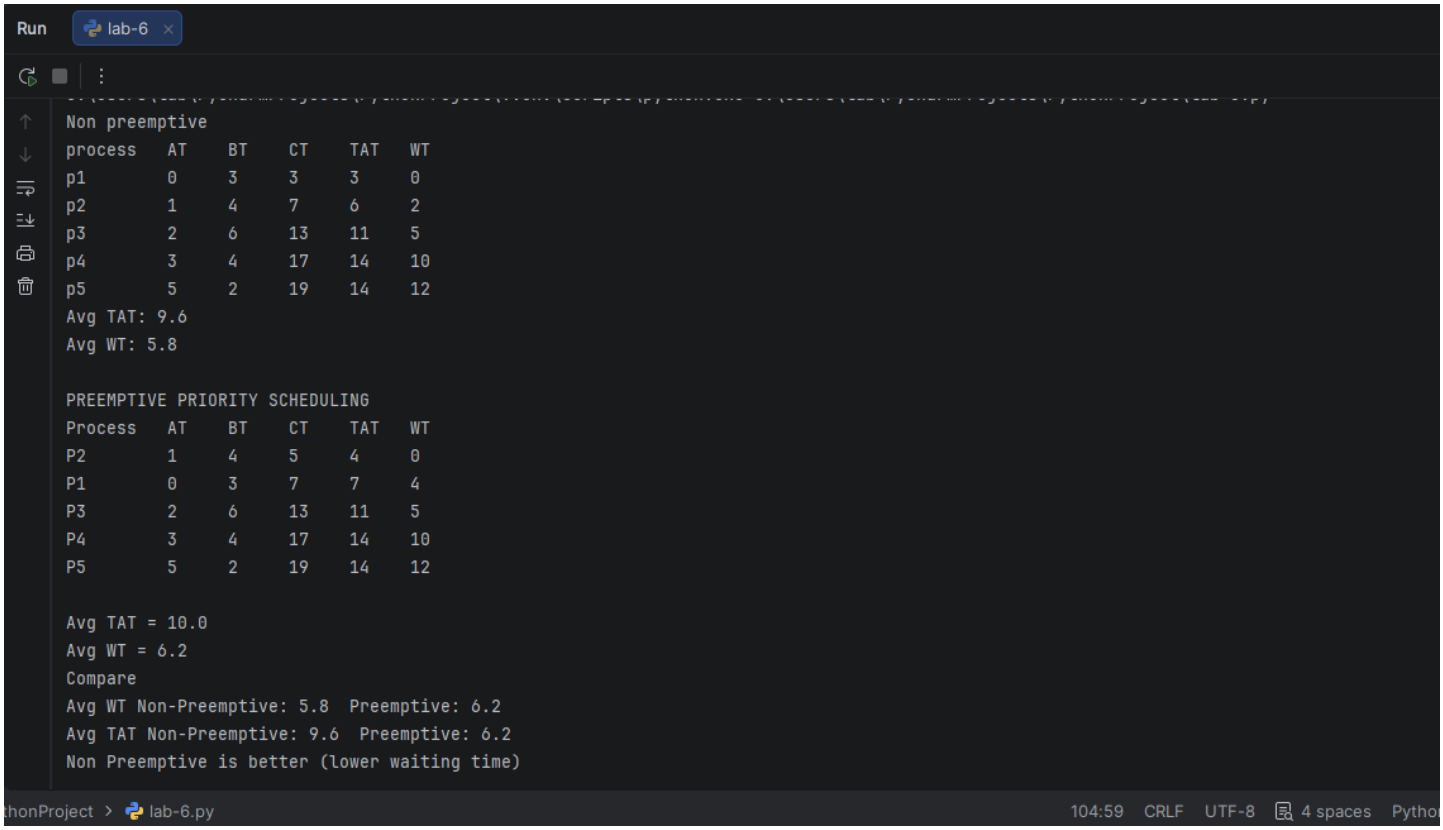
2. Source Code:

```
print("Non preemptive")

processes = [
    {'name': 'p1', 'AT':0, 'BT':3, 'priority': 3},
    {'name': 'p2', 'AT':1, 'BT':4, 'priority': 2},
    {'name': 'p3', 'AT':2, 'BT':6, 'priority': 4},
    {'name': 'p4', 'AT':3, 'BT':4, 'priority': 6},
    {'name': 'p5', 'AT':5, 'BT':2, 'priority': 10},
]

time = 0
completed = []
n = len(processes)

while len(completed) < n:
    available = [p for p in processes if p['AT'] <= time and p
not in completed]

    if not available:
        time = time + 1
        continue

    current = min(available, key=lambda p: p['priority'])

    start = time
    time += current['BT']
    end = time

    current['CT'] = end
    current['TAT'] = end - current['AT']
    current['WT'] = current['TAT'] - current['BT']
    completed.append(current)


print(f"{'process':<10}{'AT':<6}{'BT':<6}{'CT':<6}{'TAT':<6}{'
WT':<6}")
for p in completed:

print(f"{p['name']:<10}{p['AT']:<6}{p['BT']:<6}{p['CT']:<6}{p[
'TAT']:<6}{p['WT']:<6}")

avg_tat = sum(p['TAT'] for p in completed) / n
avg_wt = sum(p['WT'] for p in completed) / n
```

```
print("Avg TAT:", avg_tat)
print("Avg WT:", avg_wt)

processes = {
    'P1': {'AT': 0, 'BT': 3, 'priority': 3, 'remaining': 3},
    'P2': {'AT': 1, 'BT': 4, 'priority': 2, 'remaining': 4},
    'P3': {'AT': 2, 'BT': 6, 'priority': 4, 'remaining': 6},
    'P4': {'AT': 3, 'BT': 4, 'priority': 6, 'remaining': 4},
    'P5': {'AT': 5, 'BT': 2, 'priority': 10, 'remaining': 2},
}

time = 0
completed = {}
n = len(processes)

while len(completed) < n:

    available = [p for p, d in processes.items() if d['AT'] <=
time and d['remaining'] > 0]

    if not available:
        time += 1
        continue


    current = min(available, key=lambda p:
processes[p]['priority'])


    processes[current]['remaining'] -= 1
    time += 1


    if processes[current]['remaining'] == 0:
        at, bt = processes[current]['AT'],
processes[current]['BT']
        ct = time
        tat = ct - at
        wt = tat - bt
        completed[current] = {'AT': at, 'BT': bt, 'CT': ct,
'TAT': tat, 'WT': wt}

print(" ")
print("PREEMPTIVE PRIORITY SCHEDULING")
print(f"{'Process':<10}{'AT':<6}{'BT':<6}{'CT':<6}{'TAT':<6}{'
WT':<6}")
```

```
for p, d in completed.items():

print(f"{p:<10}{d['AT']:<6}{d['BT']:<6}{d['CT']:<6}{d['TAT']:<
6}{d['WT']:<6}")

avg_tat1 = sum(d['TAT'] for d in completed.values()) / n
avg_wt1 = sum(d['WT'] for d in completed.values()) / n
print(f"\nAvg TAT = {avg_tat1}")
print(f"Avg WT = {avg_wt1}")

print("Compare")


avg_wt_non_preemptive = avg_wt
avg_wt_preemptive = avg_wt1

avg_tat_non_preemptive = avg_tat
avg_tat_preemptive = avg_wt1

print("Avg WT Non-Preemptive:", avg_wt_non_preemptive, "
Preemptive:", avg_wt_preemptive)
print("Avg TAT Non-Preemptive:", avg_tat_non_preemptive, "
Preemptive:", avg_tat_preemptive)

if avg_wt_non_preemptive < avg_wt_preemptive:
    print("Non Preemptive is better (lower waiting time)")
else:
    print("Preemptive is better (lower waiting time)")
```

3. Output:


Run
lab-6
Non preemptive
process AT BT CT TAT WT
p1 0 3 3 3 0
p2 1 4 7 6 2
p3 2 6 13 11 5
p4 3 4 17 14 10
p5 5 2 19 14 12
Avg TAT: 9.6
Avg WT: 5.8
PREEMPTIVE PRIORITY SCHEDULING
Process AT BT CT TAT WT
P2 1 4 5 4 0
P1 0 3 7 7 4
P3 2 6 13 11 5
P4 3 4 17 14 10
P5 5 2 19 14 12
Avg TAT = 10.0
Avg WT = 6.2
Compare
Avg WT Non-Preemptive: 5.8 Preemptive: 6.2
Avg TAT Non-Preemptive: 9.6 Preemptive: 6.2
Non Preemptive is better (lower waiting time)
thonProject > lab-6.py
104:59 CRLF UTF-8 4 spaces

4. Discussion:

In this experiment, **Priority Non-Preemptive** and **Priority Preemptive CPU Scheduling** were implemented using the same group of processes. Their performances were compared by analyzing the **Average Turnaround Time (TAT)** and **Average Waiting Time (WT)**.

For **Priority Non-Preemptive Scheduling**, the obtained Average TAT was **9.6**, while the Average WT was **5.8**. In this method, a process that gets the CPU keeps running until it finishes, even if another process with a higher priority arrives during its execution.

For **Priority Preemptive Scheduling**, the Average TAT was **10.0** and the Average WT was **6.2**. Here, the currently running process may be stopped when a new process with a higher priority enters the ready queue.

From the experimental results, the **Non-Preemptive Priority algorithm** achieved better values for both Average TAT and Average WT. Hence, for the given set of processes, the Non-Preemptive approach provided better overall performance.

5. Conclusion:

The experiment demonstrated the implementation and performance comparison of **Preemptive and Non-Preemptive Priority Scheduling**. The results show that the **Non-Preemptive Priority algorithm** performed more efficiently for the selected processes, with an Average TAT of **9.6** and an Average WT of **5.8**.

In contrast, the **Preemptive Priority algorithm** resulted in an Average TAT of **10.0** and an Average WT of **6.2**. Since both performance measures were lower in the Non-Preemptive method, it can be considered the better choice for this particular dataset.

However, the Preemptive Priority approach has an advantage in situations where a high-priority process arrives and requires CPU service without much delay.

6. **GitHub:** https://github.com/AbirOPS2/CSE402-Operating-System-Lab/tree/main/Lab-06-Priority-Scheduling-NonPreemptive-Preemptive